FRIEDRICH-SCHILLER-
UNIVERSITÄT
JENA

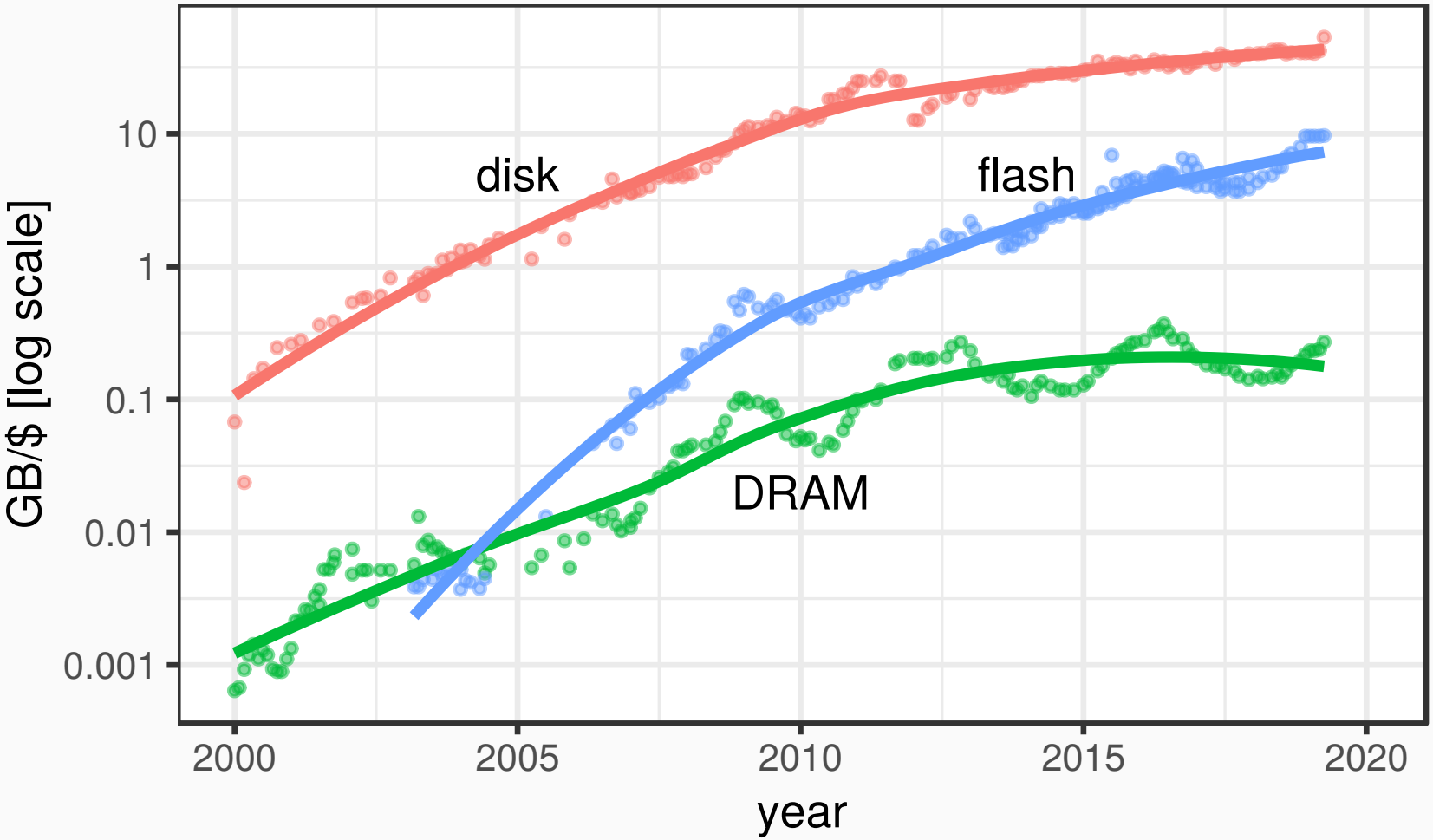
Exploiting Directly-Attached NVMe Arrays

Gabriel Haas, Michael Haubenschild[†], Viktor Leis

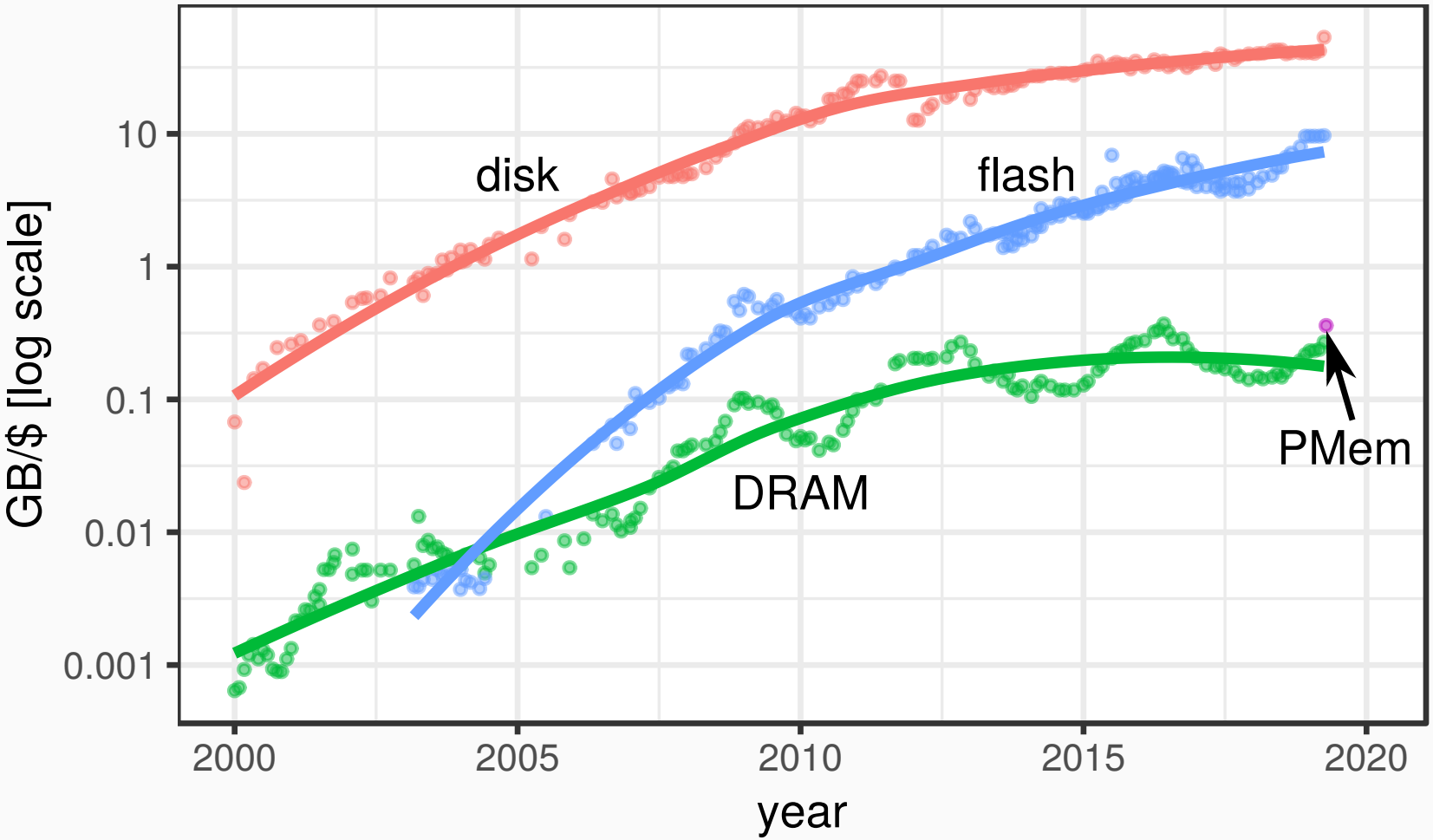
Friedrich-Schiller-Universität Jena

Tableau Software[†]

Flash is Cheap

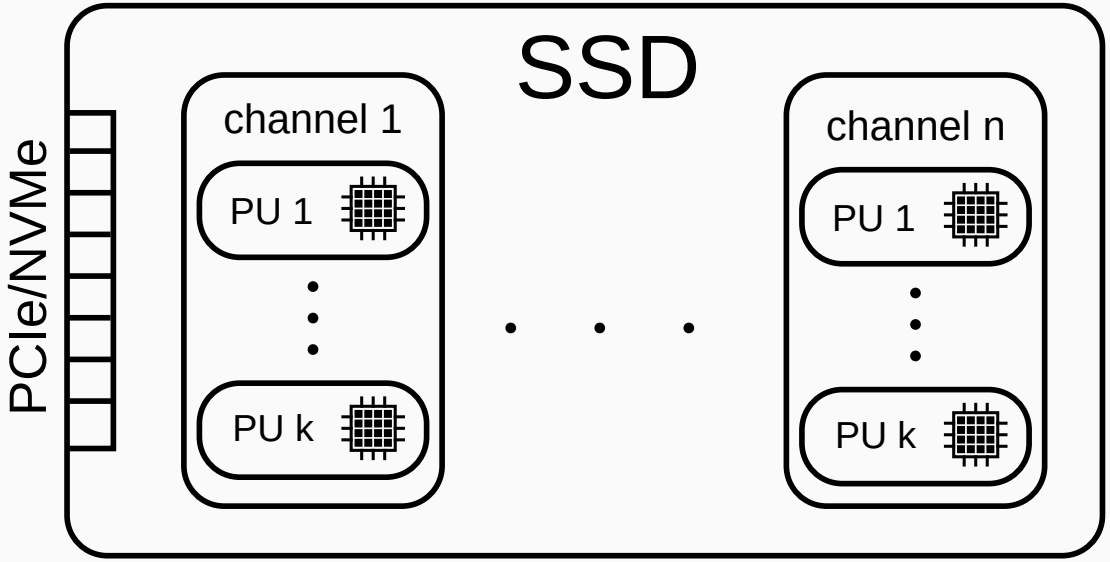


Persistent Memory is Not Cheap (Yet)



Flash is Fast

- SSDs consist of many flash chips, easy to scale bandwidth
- for a long time, SATA interface has been the bottleneck
- NVMe SSDs connect through PCIe
- Intel Skylake SP: 44 PCIe 3.0 lanes
- AMD Rome: 128 PCIe 4.0 lanes



Directly-Attached NVMe Array (DANA)

- 4, 8, or 16 PCIe-attached SSDs in one server
- PCIe 3.0: theoretical bandwidth of 12, 24, or 48 GB/s
- PCIe 4.0: theoretical bandwidth of 24, 48, or 96 GB/s
- price per GB is 20 times lower than DRAM (and PMem)
- AWS EC2 already offers instances with up to 8 NVMe SSDs

What Do $2000 Buy?

	configuration	capacity	bandwidth		random
			read	write	reads
		[TB]	[GB/s]		[M/s]
disk	20×4 TB	72.8	2	2	0.002
DANA	8×1 TB	7.3	25	20	4
pers. mem.	3×128 GB	0.4	37	10	155
DRAM	12×32 GB	0.4	92	80	1,500

Directly-Attached NVMe Array (DANA)

- economically, DANA seems like an good idea for analytics, OLAP, and HTAP
- hardware is available (and works)
- How about the software?

Experimental Setup

- 8 core/16 hyperthread Intel Coffee Lake CPU
- 4× Samsung 970 EVO Plus 1TB (13 GB/s bandwidth), RAID 0
- *fio* benchmarking tool on Linux
- complex HTAP-like I/O workload:

	I/O	I/O depth	rate limit
scan	read	128	-
point read	read	1	-
bg. write	write	1	250 MB/s
WAL	write+sync	1	250 MB/s

Baseline (The PostgreSQL Way of Doing I/O)

- synchronous I/O using `pread`, `pwrite`, `fdatasync`
- OS page buffer enabled
- ext4 file system
- 128 scan threads
- 1 point read thread
- 1 background writer thread
- 1 WAL writer thread

Baseline Results

description	scan MB/s	point MB/s	bg. wr. MB/s	WAL MB/s	total MB/s	CPU cycl./byte
baseline	3,503	27	250	6	3,786	6.92

WAL writer extremely slow due to `fdatasync`

Moving `fdatasync` Off the Critical Path

description	scan MB/s	point MB/s	bg. wr. MB/s	WAL MB/s	total MB/s	CPU cycl./byte
baseline	3,503	27	250	6	3,786	6.92
− `fdatasync`	5,055	39	250	250	5,595	10.52

- we are CPU bound again, this time due to the Linux kernel!
- CPU cycles are needed for query processing rather than I/O
- using 128+3 threads seems excessive

Asynchronous I/O

description	scan MB/s	point MB/s	bg. wr. MB/s	WAL MB/s	total MB/s	CPU cycl./byte
baseline	3,503	27	250	6	3,786	6.92
− `fdatasync`	5,055	39	250	250	5,595	10.52
+ async. I/O	3,439	118	252	251	4,059	7.61

- we are still wasting lots of CPU
- and the scan got much got slower
- maybe the OS page buffer is the problem?

Disabling OS Page Buffer

description	scan MB/s	point MB/s	bg. wr. MB/s	WAL MB/s	total MB/s	CPU cycl./byte
baseline	3,503	27	250	6	3,786	6.92
− `fdatasync`	5,055	39	250	250	5,595	10.52
+ async. I/O	3,439	118	252	251	4,059	7.61
+ `O_DIRECT`	4,464	74	250	249	5,036	1.01

- CPU utilization is reasonable now
- result are ok, but total throughput still lower than before
- we are still using a file system (ext4)

Raw Block Device Access

description	scan MB/s	point MB/s	bg. wr. MB/s	WAL MB/s	total MB/s	CPU cycl./byte
baseline	3,503	27	250	6	3,786	6.92
− `fdatasync`	5,055	39	250	250	5,595	10.52
+ async. I/O	3,439	118	252	251	4,059	7.61
+ `O_DIRECT`	4,464	74	250	249	5,036	1.01
− file system	5,443	52	251	251	5,997	0.76

- performance good, but we had to disable all OS features

File System

file system	scan MB/s	point MB/s	write MB/s	total I/O MB/s	CPU cycl./byte
ext4	4,464	74	499	5,036	1.01
XFS	5,395	53	502	5,950	0.79
Btrfs	3,364	94	500	3,957	2.42
F2FS	4,786	67	504	5,358	0.99
block	5,443	52	502	5,997	0.76

Many More Experiments in the Paper

more I/O stack experiments:

- software RAID 5
- user-space I/O with SPDK

flash peculiarities:

- impact of the page size
- latency experiments
- SLC cache
- garbage collection
- enterprise vs. consumer SSDs

Implications of DANA on Database Systems

- page-wise storage necessary (e.g., B-trees)
- flash latency (100us) requires DRAM caching
- high-performance storage engines are necessary
- combine techniques of in-memory systems with a high-level architecture more similar to disk-based systems

LeanStore on DANA (Preliminary TPC-C Results)

	ktxn/s	read GB/s	write GB/s
PostgreSQL in-mem.	10	0.1	0.05
PostgreSQL oom.	9	0.2	0.07
LeanStore in-mem.	410	0	0.8
LeanStore oom.	130	0.9	1.4

Recommendations

- avoid (`fsync`), use persistent memory for logging
- bypass OS buffer pool (`O_DIRECT`)
- asynchronous I/O
- avoid file system or use XFS

Conclusions

- DANA is cheap
- DANA is fast
- very interesting for large-scale analytics
- operating system I/O stack is now a bottleneck
- new storage engines are necessary

`http://leanstore.io`